

#6 JDBC

JDBC Fundamentals	JDBC architecture, drivers, database connections, PreparedStatement, ResultSet processing
JDBC CRUD Operations	Implementing CRUD operations using Java and JDBC, transaction basics

Java Database Connectivity

Introduction

Java Database Connectivity (JDBC) is an **Application Programming Interface (API)** in Java that allows applications to communicate with relational databases. JDBC acts as a bridge between Java applications and database management systems (DBMS), enabling developers to perform operations such as **inserting, retrieving, updating, and deleting (CRUD operations)** data from databases using SQL queries.

JDBC provides a **standardized way** to interact with databases, ensuring that Java applications remain **portable** across different database systems without significant modifications.

Purpose and Architecture of JDBC

Purpose of JDBC

JDBC (Java Database Connectivity) is a standard Java API that enables Java applications to interact with relational databases. It provides a uniform interface for accessing different databases, allowing developers to write database-independent applications.

The primary purpose of JDBC is to establish a connection between Java programs and databases, execute SQL statements, and retrieve and manipulate data stored in databases. JDBC eliminates the need for database-specific coding by providing a common set of interfaces and classes.

JDBC ensures portability, scalability, and maintainability of Java applications by allowing the same Java code to work with multiple databases by simply changing the JDBC driver.

Objectives of JDBC

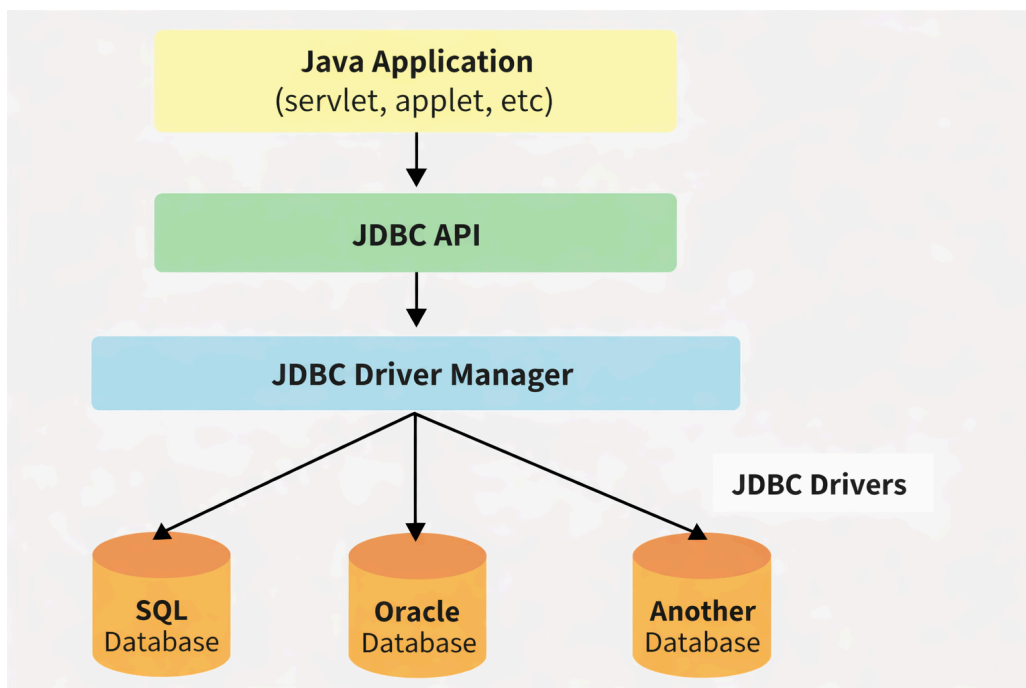
- To provide a standard API for database access in Java

- To enable database-independent application development
- To execute SQL queries and updates from Java programs
- To manage database connections efficiently
- To retrieve and process query results in Java applications

JDBC Architecture

JDBC architecture follows a layered approach that separates Java applications from database-specific implementations. It consists of multiple components that work together to enable database communication.

Components of JDBC Architecture



1. Java Application

The Java application represents the client program that requires access to the database. It uses JDBC API classes and interfaces to interact with the database.

2. JDBC API

The JDBC API provides a set of standard interfaces and classes defined in the **java.sql** and **javax.sql** packages. These interfaces allow Java applications to perform database operations such as establishing connections, executing SQL statements, and processing results.

Key interfaces include:

- Driver
- Connection
- Statement
- PreparedStatement
- CallableStatement
- ResultSet

3. DriverManager

DriverManager is responsible for managing JDBC drivers. It selects the appropriate driver based on the database URL and establishes a connection between the Java application and the database.

4. JDBC Driver

A JDBC driver is a vendor-specific implementation that converts JDBC method calls into database-specific requests. It acts as a communication link between the Java application and the database.

5. Database

The database is the backend system where data is stored. It executes SQL queries received from the JDBC driver and returns results to the Java application.

Working of JDBC Architecture

1. The Java application sends a request using JDBC API
 2. DriverManager identifies and loads the appropriate JDBC driver
 3. The JDBC driver establishes a connection with the database
 4. SQL statements are sent to the database for execution
 5. The database processes the request and returns the result
 6. The result is delivered back to the Java application
-

Advantages of JDBC Architecture

- Database independence
 - Platform independence
 - Ease of maintenance
 - Secure and efficient database access
 - Supports enterprise-level applications
-

JDBC Drivers

JDBC Drivers are **software components** that allow Java applications to connect to databases. They act as a bridge between a **Java application and a database management system (DBMS)** by converting Java calls into database-specific operations.

Each database (MySQL, PostgreSQL, Oracle, SQL Server, etc.) requires a compatible JDBC driver to communicate with it. JDBC drivers differ in how they handle this communication, which leads to different performance levels and use cases.

There are **four types of JDBC drivers**, each with its own advantages and limitations.

1. JDBC-ODBC Bridge Driver (Type 1)

The **JDBC-ODBC Bridge Driver** uses the **Open Database Connectivity (ODBC)** driver to interact with databases. ODBC is a standard API that allows applications to access database management systems (DBMSs) regardless of the database vendor.

How It Works

- The Java application sends a database request to the JDBC-ODBC Bridge Driver.
- The bridge translates the request into an ODBC call.
- The ODBC driver communicates with the database and retrieves the requested data.
- The data is then sent back to the Java application.

Characteristics

- ✓ Works with any database that has an **ODBC driver**.
- ✓ Can be used when there is no direct JDBC driver available for a database.

- ✗ **Requires ODBC Setup**, which can be complex.
- ✗ **Slower performance** due to multiple translation layers.
- ✗ **Not platform-independent** since it relies on native ODBC drivers.
- ✗ **Deprecated in Java 8** and removed in Java 9 due to security risks and inefficiency.

Example Usage (Before Deprecation)

```
import java.sql.Connection; // Ummed Singh
import java.sql.DriverManager;
import java.sql.SQLException;

public class Type1JDBCExample {

    public static void main(String[] args) {

        try {

            Class.forName("sun.jdbc.odbc.JdbcOdbcDriver"); //
            Load JDBC-ODBC Driver

            Connection con =
            DriverManager.getConnection("jdbc:odbc:myDSN", "user",
            "password");

            System.out.println("Connected using Type 1
            Driver");

            con.close();

        } catch (Exception e) {

            e.printStackTrace();

        }

    }

}
```

⚠ **This code will not work in Java 8+** because the JDBC-ODBC Bridge has been removed.

2. Native API Driver (Type 2)

The **Native API Driver** interacts with the database using **database vendor-specific native libraries**. It does not require an ODBC driver but depends on the **database's native client software** to function.

How It Works

- The Java application calls the Type 2 driver.
- The driver translates JDBC calls into **native API calls** provided by the database vendor.
- The database processes the request and returns the result via the **native API**.

Characteristics

- ✓ **Better performance than Type 1** since it avoids ODBC overhead.
- ✓ Works well when the native library is optimized for a particular database.
- ✗ **Requires native database libraries**, making it **platform-dependent**.
- ✗ Must be installed **on every client machine**, increasing complexity.
- ✗ Not suitable for web applications where clients may use different platforms.

Example Usage (Oracle Type 2 Driver)

```
import java.sql.Connection; // Ummed Singh
import java.sql.DriverManager;
import java.sql.SQLException;

public class Type2JDBCExample {

    public static void main(String[] args) {

        try {

            Class.forName("oracle.jdbc.driver.OracleDriver"); //
            Oracle Native Driver

            Connection con =
            DriverManager.getConnection("jdbc:oracle:oci8:@localhost:1521:xe
            ", "user", "password");
```

```
        System.out.println("Connected using Type 2 Driver");  
  
        con.close();  
  
    } catch (Exception e) {  
  
        e.printStackTrace();  
  
    }  
  
}  
  
}
```

 Requires Oracle's native OCI (Oracle Call Interface) libraries installed on the system.

3. Network Protocol Driver (Type 3)

The **Network Protocol Driver** (also called **Middleware Driver**) communicates with the database through a **middleware server**, which forwards the requests to the appropriate database.

How It Works

- The Java application sends JDBC calls to the Type 3 driver.
- The driver forwards these calls to a **middleware server** over the network.
- The middleware translates them into **database-specific calls** and communicates with the actual database.
- The database processes the request and sends the result back through the middleware.

Characteristics

- ✓ **More scalable** than Type 1 and Type 2.
- ✓ **Database-independent** – The middleware handles different database protocols.
- ✓ Suitable for **enterprise applications** with centralized database management.
- ✗ **Requires an additional middleware server**, adding overhead.
- ✗ Performance depends on **network latency and middleware efficiency**.

Example Usage

Middleware solutions like **IBM WebSphere** and **Sybase** use Type 3 drivers.

```
import java.sql.Connection; // Ummed Singh
import java.sql.DriverManager;
import java.sql.SQLException;

public class Type3JDBCExample {

    public static void main(String[] args) {

        try {

            Connection con =
                DriverManager.getConnection("jdbc:net://middleware-server:1
                521/mydb", "user", "password");

            System.out.println("Connected using Type 3
            Driver");

            con.close();

        } catch (Exception e) {

            e.printStackTrace();

        }

    }

}
```

📌 Rarely used today because **Type 4 drivers** provide better performance without middleware.

4. Thin Driver (Type 4)

The **Thin Driver**, also called the **Pure Java Driver**, is the most commonly used JDBC driver today. It directly communicates with the database using its **native protocol** over the network.

How It Works

- The Java application sends JDBC calls to the Type 4 driver.
- The driver directly interacts with the database over TCP/IP using the database's **native communication protocol**.
- The database processes the request and returns the result directly to the application.

Characteristics

- ✓ **Fastest and most efficient** since it eliminates extra translation layers.
 - ✓ **Pure Java implementation**, making it **platform-independent**.
 - ✓ **No additional software required**, making deployment easier.
 - ✓ The **preferred choice** for most modern Java applications.
- ✗ Requires **different drivers for different databases** (e.g., MySQL, PostgreSQL, Oracle, etc.).

Example Usage (MySQL Type 4 Driver)

```
import java.sql.Connection; // Ummed Singh
import java.sql.DriverManager;

public class Type4JDBCExample {

    public static void main(String[] args) {

        try {

            Class.forName("com.mysql.cj.jdbc.Driver"); // MySQL
            Thin Driver

            Connection con =
            DriverManager.getConnection("jdbc:mysql://localhost:3306/my
            db", "root", "password");

            System.out.println("Connected using Type 4
            Driver");

            con.close();
```

```
    } catch (Exception e) {  
  
        e.printStackTrace();  
  
    }  
  
}  
  
}
```

📌 Most modern databases provide **Type 4 JDBC drivers**, such as **PostgreSQL, MySQL, Oracle, and SQL Server**.

Steps to Connect to a Database using JDBC

To interact with a database using JDBC, a Java application follows a sequence of well-defined steps. These steps ensure secure and efficient communication between the Java program and the database.

Step 1: Load and Register the JDBC Driver

The JDBC driver acts as a communication link between the Java application and the database. Loading the driver registers it with the **DriverManager**.

This step ensures that the appropriate database driver is available to establish a connection.

Step 2: Establish a Connection

A connection represents a session between the Java application and the database.

The **DriverManager** class is used to create this connection using database URL, username, and password.

Once the connection is established, the Java application can send SQL commands to the database.

Step 3: Create a Statement Object

A **Statement** object is used to send SQL queries to the database.

It allows execution of static SQL statements such as SELECT, INSERT, UPDATE, and DELETE.

The statement object works on the established database connection.

Step 4: Execute SQL Query

SQL queries are executed using the statement object.

Depending on the type of SQL command, the execution may return a result or the number of affected rows.

Step 5: Process the Result

When a SELECT query is executed, the database returns data in the form of a **ResultSet**.

The **ResultSet** object stores the result of the query and allows the application to read data row by row.

Step 6: Close the Connection

After completing database operations, all resources such as **ResultSet**, **Statement**, and **Connection** must be closed to avoid memory leaks.

DriverManager

DriverManager is a class that manages a list of database drivers.

It selects the appropriate driver based on the database URL and establishes a connection between the Java application and the database.

Functions of DriverManager:

- Registers JDBC drivers
- Establishes database connections
- Manages multiple drivers

Connection

A **Connection** object represents a physical connection to the database. It acts as a gateway through which SQL statements are sent to the database.

Key responsibilities of Connection:

- Creates **Statement** and **PreparedStatement** objects
 - Manages transactions
 - Maintains session with the database
-

Statement

The **Statement** interface is used to execute simple SQL statements without parameters.

Characteristics:

- Used for static SQL queries
 - SQL query is compiled every time it is executed
 - Suitable for simple and one-time queries
-

PreparedStatement

PreparedStatement is a subinterface of **Statement** used for executing parameterized SQL queries.

Characteristics:

- SQL query is precompiled
- Supports dynamic parameters
- Improves performance
- Protects against SQL Injection

PreparedStatement is preferred when the same query is executed multiple times with different values.

ResultSet

ResultSet is an object that holds the data returned by a SELECT query. It acts like a table of data that can be traversed row by row.

Features of ResultSet:

- Maintains cursor position
- Retrieves data using column name or index
- Allows sequential access to query results

CRUD Operations Using JDBC

CRUD stands for **Create, Read, Update, and Delete**. These operations are performed using JDBC with the following steps:

1. Establishing a Database Connection

```
import java.sql.Connection; // Ummed Singh
import java.sql.DriverManager;

public class DatabaseConnection {
    public static Connection getConnection() {
        Connection con = null;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb",
"root", "password");
            System.out.println("Connection Established
Successfully");
        } catch (Exception e) {
            e.printStackTrace();
        }
        return con;
    }
}
```

2. Creating a Table in Database

```
import java.sql.Connection; // Ummed Singh
import java.sql.Statement;
public class CreateTable { // Ummed Singh
    public static void main(String[] args) {
        try {
            Connection con = DatabaseConnection.getConnection();
            Statement stmt = con.createStatement();
            String sql = "CREATE TABLE students (id INT PRIMARY
KEY, name VARCHAR(50), age INT)";
            stmt.executeUpdate(sql);
            System.out.println("Table Created Successfully");
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

3. Insert Data into Database (Create Operation)

```
import java.sql.Connection; // Ummed Singh
import java.sql.PreparedStatement;
public class InsertData {
    public static void main(String[] args) {
        try {
            Connection con = DatabaseConnection.getConnection();
            String query = "INSERT INTO students (id, name, age)
VALUES (?, ?, ?)";
            PreparedStatement pstmt =
con.prepareStatement(query);
            pstmt.setInt(1, 1);
            pstmt.setString(2, "Ummed Singh");
            pstmt.setInt(3, 24);
            pstmt.executeUpdate();
            System.out.println("Data Inserted Successfully");
            con.close();
        }
    }
}
```

```
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

4. Fetch Data from Database (Read Operation)

```
import java.sql.Connection; // Ummed Singh  
import java.sql.ResultSet;  
import java.sql.Statement;  
public class ReadData {  
    public static void main(String[] args) {  
        try {  
            Connection con = DatabaseConnection.getConnection();  
            Statement stmt = con.createStatement();  
            ResultSet rs = stmt.executeQuery("SELECT * FROM  
students");  
  
            rs.getInt("id");  
            while (rs.next()) {  
                System.out.println("ID: " + rs.getInt("id") + ",  
Name: " + rs.getString("name") + ", Age: " + rs.getInt("age"));  
            }  
            con.close();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

5. Update Data in Database (Update Operation)

```
import java.sql.Connection; // Ummed Singh  
import java.sql.PreparedStatement;  
public class UpdateData {  
    public static void main(String[] args) {  
        try {  
            Connection con = DatabaseConnection.getConnection();
```

```
        String query = "UPDATE students SET age = ? WHERE id  
= ?";  
        PreparedStatement pstmt =  
con.prepareStatement(query);  
        pstmt.setInt(1, 22);  
        pstmt.setInt(2, 1);  
        pstmt.executeUpdate();  
        System.out.println("Data Updated Successfully");  
        con.close();  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}  
}
```

6. Delete Data from Database (Delete Operation)

```
import java.sql.Connection; // Ummed Singh  
import java.sql.PreparedStatement;  
  
public class DeleteData {  
    public static void main(String[] args) {  
        try {  
            Connection con = DatabaseConnection.getConnection();  
            String query = "DELETE FROM students WHERE id = ?";  
            PreparedStatement pstmt =  
con.prepareStatement(query);  
            pstmt.setInt(1, 1);  
            pstmt.executeUpdate();  
            System.out.println("Data Deleted Successfully");  
            con.close();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Non-Conventional Database

Non-conventional databases, also known as **NoSQL databases**, are different from traditional relational databases (like MySQL, Oracle, PostgreSQL). Instead of storing data in tables with rows and columns, they use **different data storage techniques** such as **document-based, key-value, column-family, or graph-based models**.

♦ **Examples of Non-Conventional Databases:**

- ✓ **MongoDB** → Document-based (NoSQL).
- ✓ **Cassandra** → Column-family-based (NoSQL).
- ✓ **Firebase** → Cloud-based real-time database.
- ✓ **Redis** → Key-value store (often used for caching).

These databases are **schema-less, scalable, and flexible**, and are great for handling **unstructured or semi-structured** data.

JDBC (Java Database Connectivity) **cannot be used directly** with these databases. Instead, Java provides **custom drivers and APIs** for connecting to them.

What Problems Do NoSQL Databases Solve?

Problem	Solution by NoSQL
Rigid schema	NoSQL is schema-less – different documents can have different structures.
Performance bottlenecks	Designed for high-speed read/write operations.
Scalability	Scales easily across multiple servers (horizontal scaling).
Handling Big Data	Excellent for storing and querying massive datasets.
Complex relationships	Graph databases handle relationships efficiently.

Introduction to MongoDB

MongoDB is a **NoSQL database** that stores data in **JSON-like documents** (BSON format). It is used for applications that require **scalability and flexibility**. A typical MongoDB document looks like this:

```
{  
  "_id": "12345",  
  "name": "Ummed Singh",  
  "email": "Ummed@example.com",  
  "age": 24  
}
```

MongoDB collections are like tables, and documents are like rows, but there is no strict schema.

♦ Structure

- **Database** → contains **collections**
- **Collection** → contains **documents**
- **Document** → stores the actual data (like a row in SQL)

byte^{XL}